

Sail Extensions: A Design Review

Contracts, alternatives, and a complete Sedona example

Querygraph

2026-10-01

Contents

Sail extensions: the first contract	3
Three decisions, in order	3
A thirty-minute review	3
From the brief to the design	4
The eight-module map	4
The decision map	4
How this fits Sail's design	5
1. Discovery and binding	6
2. Functions	7
3. Relations	8
4. Commands and receipts	9
5. Placement and replay	10
6. Native memory	11
7. Lifecycle and teardown	12
8. Packaging and compatibility	13
Complete code and reading path	14
The small author sample	14
Running the sample	14
Listings	14

Sail extensions: the first contract

This review proposes a small, independently packaged native extension surface for Sail. The first decision is whether to accept **scalar functions discovered from Python wheels, exchanged through DataFusion FFI and resolved on workers**. Stateful relations and distributed native operators are subsequent decisions. The prototype demonstrates useful behavior; its implementation is evidence for review, not a request to freeze all eight modules as public API.

The upstream maintainer's direction is already the starting point: Python distribution, DataFusion FFI, independent release schedules and a narrow contract that leaves Sail free to refactor. Spark Connect describes client requests; the FFI connects native packages to the engine. These are complementary boundaries. Sail retains planning, scheduling and resource policy. The design question is which information an extension must expose so those responsibilities remain possible. [Upstream discussion](#)

Three decisions, in order

1. **Accept a scalar foundation.** Specify discovery, compatibility rejection, names, schema metadata, owner lifetime and worker package identity. Accept it when the small Sedona example composes with ordinary SQL across a shuffle, and incompatible or missing packages fail clearly.
2. **Review optional stateful relations.** Specify DataFrame inputs, planning without mutations, execution receipts, memory leases and cancellation. Require all-input consumption, explicit indeterminate outcomes and final-owner accounting. Driver gathering is the prototype's capacity boundary.
3. **Design distributed native operators.** Evaluate a focused spatial-join hook against broader planner hooks. Separate stateless worker execution from persistent graph ownership and recovery. Require truthful distribution, ordering, effects and resource requirements before promising scalable native execution.

The public contract should state observable behavior and ownership. Wrapper names, session-manager organization, one-partition adapters and prepaid quota sizes remain replaceable implementation choices. Exact engine-version pins qualify this prototype; they do not promise compatibility with every future DataFusion or Sail release.

A thirty-minute review

Spend five minutes here and on the [decision map](#), fifteen on the [complete Sedona sample](#), and ten choosing the first acceptance boundary. The main sample is 246 lines of Rust, Python bootstrap and packaging plus the existing 20-line client. Build support, tests and protocol/resource interfaces follow in full listings.

The example exports native scalar functions. Indexed spatial joins, session SET propagation and the complete Sedona feature set remain outside it. No new runtime or cluster qualification is claimed by this document build.

For a consecutive deeper reading, follow: [Module 1: Discovery](#) → [Module 2: Functions](#) → [Module 3: Relations](#) → [Module 4: Commands](#) → [Module 5: Placement](#) → [Module 6: Memory](#) → [Module 7: Lifecycle](#) → [Module 8: Compatibility](#). Each chapter separates the proposed contract, current implementation, alternatives and acceptance criteria.

Review target: `querygraph/sail` at `bd8ce9ae8839477e2c08a0475ab7900b115c5366`. Later graph experiments and optimizations are outside this snapshot. Continue with the [module and decision maps](#). The separate [host companion](#) compares plain Sail with the prototype and maps changes to their proposed owners.

From the brief to the design

The brief asks for three decisions. The following maps connect those decisions to the eight parts of the existing prototype. Read them as an index of review responsibilities: an implemented component can establish feasibility without becoming part of the stable author-facing contract.

The eight-module map

This retains Fable's module map, with the scope and choices made explicit. The rows also provide the consecutive reading order.

Module	Owner	Prototype status	Choice to review
Module 1: Discovery and binding	Sail session integration; package author	Flag-gated Python entry points and per-session binding	Wheel discovery plus FFI, versus statically linked embedding or another package loader
Module 2: Functions	Extension; Sail registry and codec	Native scalar functions, including workers	Names, collisions, metadata and identity; later specialized planning needs a separate decision
Module 3: Relations	Sail Connect/planner; extension handler	Bounded envelope, driver-local provider	Explicit DataFrame inputs versus table-function or catalog-based invocation
Module 4: Commands	Extension execution; host outcome handling	Mutating relations emit receipts	Reuse the relation path versus a dedicated command entry point; effects remain explicit
Module 5: Placement and replay	Sail scheduler; extension declarations	Driver-native regions receive one attempt	Separate placement, state ownership and replay safety; preserve a path to worker operators
Module 6: Native memory	Sail admission; participating extension	Prepaid, non-spillable driver quota	Shared admission and lease lifetime versus the particular prepayment policy
Module 7: Lifecycle and teardown	Sail session/executor lifecycle; native owner	Cleanup corrections; stuck-owner policy open	Cooperative cancellation, retained ownership and a bounded operational failure policy
Module 8: Packaging and compatibility	Package author; host/worker validation	Exact version pins and content identities	Initial tested combinations versus a future compatibility promise

The scalar foundation primarily needs modules 1, 2 and 8, plus the applicable ownership and cancellation behavior in module 7. The stateful relation surface adds modules 3 to 6. This is a capability split, not a claim that memory or lifecycle correctness can be omitted from scalar execution. Some generic codec and lifecycle corrections apply with extension discovery disabled.

The decision map

This second table turns the implementation inventory into questions that can change its shape. It preserves the review's comparison while identifying the cost of each choice.

Topic	What the prototype establishes	Decision and trade-off
Minimum contract	A set of host adapters can support independent packages.	Promise schemas, ownership and errors; keep wrapper and manager structure private. A smaller ABI still needs adequate semantic information.

Topic	What the prototype establishes	Decision and trade-off
Distributed execution	Scalars travel to workers; native relations gather on the driver.	Gathering simplifies state ownership but concentrates data and work. Compare partition-preserving stateless operators separately from recoverable graph state.
Sedona integration	Native scalar predicates compose with general Sail joins.	A focused join hook can constrain complexity; broad optimizer hooks offer more freedom but expose ordering and plan internals. Neither is proved by the scalar example.
Native memory	Prepaid quota remains charged through its final owner.	Prepayment simplifies admission but holds idle capacity. Coordinated subpools or dynamic admission can preserve one authority with more bookkeeping.
Compatibility	Exact versions and package identity reject known mismatches.	Start with qualified combinations; expand only with compatibility evidence. Discovery and manifest checks do not make arbitrary native code safe.

How this fits Sail's design

The upstream maintainer's public feedback endorses Python packaging and an FFI built on DataFusion, while treating the session mutator as an internal detail. The prototype follows that direction. The original broad Rust-trait proposal is an alternative from an extension author, not evidence that Sail prefers that API. [the maintainer's comment](#)

Sail's authors also describe lakehouse operations as engine-visible plans that can be optimized and distributed. Applying that principle here is a design inference: independently shipped extensions should describe enough semantics for Sail to manage their work. An opaque driver region is a useful initial boundary, with an explicit limit; it does not establish the distributed operator model. [Engine-contract article](#)

The next chapters move from finding a package, to calling its functions, to constructing relations and effects, and then to the placement, ownership and compatibility obligations that make those operations reliable. Each ends by connecting to the next concern. The final code section gives the complete small sample and the supporting listings without interrupting that argument.

Continue to [Module 1: Discovery and binding](#).

1. Discovery and binding

The first boundary is installation: how does an independently built package become available to a Sail session? The prototype uses Python packaging for discovery and DataFusion FFI for native objects. Those are complementary choices. A wheel supplies a familiar distribution mechanism; it does not, by itself, establish binary compatibility or define query semantics.

The **public behavior** is a discoverable `pysail.extensions` entry point, a readable manifest, and a fresh bound owner for each session incarnation. The manifest names the package, version, API and engine versions, placement, and supported relation types. Binding returns the object that supplies scalar functions or relation handlers. Declared mismatches and duplicate registrations must fail before those registrations become visible. Packages requesting a native quota additionally implement `bind_with_resources`; ordinary scalar packages can use `bind`. The owner must remain alive while its callbacks or exported objects can still be used.

At the pinned Sail commit, the **implementation** checks API version 1, DataFusion 55.1.0 and Arrow 59.3.0 before importing native capsules. Discovery is enabled by `SAIL_EXPERIMENTAL_EXTENSIONS=1`. The session loader assigns an incarnation, hashes installed package files and manifest options, and retains loaded code for the process lifetime. These checks identify known mismatches; the loader still trusts the package's declared capsule layout and native callbacks. They are not a sandbox or proof of arbitrary binary safety. [Manifest validation and binding](#).

Sedona has no Sail host-crate dependency and implements no Sail Rust trait. The [Sedona listing](#) nevertheless needs its own [Python manifest](#), [entry point](#), [Cargo dependencies](#) and [build preparation](#). Nutmeg makes a different, explicit choice: its optional memory contract depends on the small [resource ABI](#) crate, `sail-native-resource-ffi`. “No host trait dependency” describes both paths more accurately than “no Sail dependency.”

A statically linked Rust trait can suit an application built as one unit; it is not the selected boundary for separately compiled wheels. A C loader could also retain Python packaging or use another discovery mechanism. Python UDFs remain useful for expression workloads, including wrappers around native code, but do not alone supply this table-provider contract. None of these alternatives is intrinsically disqualified by its language.

The acceptance questions are concrete: does an incompatible manifest fail, does a duplicate name leave no partially registered catalog, and can one session's owner outlive another without sharing mutable session state? The source provides validation tests; a platform verdict still needs the matching artifact receipts. The decision here is whether trusted wheel discovery and session ownership are the minimum supported surface. With that established, [functions](#) are the smallest useful native export.

2. Functions

[Discovery](#) gives Sail a bound package owner. The simplest use of that owner is a scalar function: the client submits an ordinary named function call, and Sail resolves it without a custom expression protocol. Sedona provides the concrete example. Its [170-line Rust adapter](#) wraps existing Apache SedonaDB Rust and GEOS kernels rather than reimplementing spatial operations in Sail.

The **public behavior** is `scalar_udfs()` returning objects whose `__datafusion_scalar_udf__()` method exports a `datafusion_scalar_udf` capsule containing `FFI_ScalarUDF`. Names, aliases, argument and return semantics, including metadata-bearing fields, must survive planning and worker execution. The package owner remains live for every exported callback. The current policy rejects collisions with built-ins or other registrations. Scalar packages use `placement: "any"`; every executing worker must have the matching installed package identity. Native function pointers do not travel over the network.

The **implementation** retains Python owners, wraps imported functions and encodes package identity plus expression fields in Sail's task codec. Those wrapper types and registry structures can change without becoming an API that extension authors implement. The semantic requirement is that a geometry's metadata survives intervening built-ins and a shuffle, and that a worker rejects an unavailable or different package instead of selecting another implementation by name.

The Sedona scope is deliberately smaller than a spatial engine. The package exports 128 native scalar functions, not the entire SedonaDB catalog. It omits five colliding functions: `st_asbinary`, `st_geomfromwkb`, `st_geogfromwkb`, `st_setsrid` and `st_srid`. Each binding captures immutable default options because the FFI configuration path does not preserve Sedona's typed runtime services; host `SET sedona.*` propagation is absent. Raster, GeoParquet readers, PROJ services and indexed `SpatialJoinExec` are not supplied. General Sail joins can use spatial predicates, but this does not demonstrate an indexed spatial join or its performance. [Exact package scope](#).

A raw `Expression.extension` would support custom expression syntax but require another client representation and dispatch contract. A linked built-in would avoid dynamic loading while coupling domain releases to the host build. Driver-only functions could serve local use, but would need an explicit placement restriction in distributed queries. Name resolution is the smallest current choice for existing DataFusion scalar kernels, not an answer to every extension shape.

Acceptance should cover nulls, aliases, geometry-field composition, worker identity mismatch and an actual shuffle; the repository contains corresponding fixtures, not a fresh execution verdict from this document. Reviewers should decide the collision and configuration policies explicitly. A function still returns values within a row: producing a new table requires the [relation boundary](#).

3. Relations

[Scalar functions](#) fit ordinary expressions. A graph algorithm or another table-producing operation instead needs a request, zero or more DataFrame inputs, and a result schema. The proposal has two boundaries: Spark Connect carries that request from the client; DataFusion FFI connects Sail to the installed implementation. Choosing the Connect envelope does not replace or compete with choosing the native FFI.

The **public behavior** is a lazy relation. `Relation.extension` contains a registered type URL and either an allowed bare payload or version 1 of `SailExtensionRequest`, with payload bytes and input plans. The host validates the request before dispatch, resolves the inputs and preserves their column names. The bound package receives execution-plan capsules and returns a `datafusion_table_provider` capsule describing its output. Planning, schema inspection and EXPLAIN must not consume inputs or perform mutations. That is an obligation of the trusted handler, not something a capsule can enforce.

The pinned **implementation** limits the envelope to 8 MiB, payload to 1 MiB, inputs to 16, input nesting depth to 64 and type URLs to 512 bytes. Input expressions are rejected. These bounds are reviewable policy, not incidental parser behavior. [Wire limits](#). The handler receives planned physical inputs, not an FFI logical-plan API or permission to replace Sail's optimizer. A host adapter retains Sail's task context and runtime and coalesces every input partition into one host input partition. Distributed native relations are driver-resident in this contract; input gathering is not distributed execution of the native algorithm. [Input adapter](#).

A registered table function is a legitimate alternative for a table produced from ordinary arguments. Arguments can carry string or binary payloads; lack of an opaque payload is not a reason to reject it. The missing convenience in the current function path is direct passage of arbitrary DataFrame plans. Named tables or another handle protocol would need their own ownership and consistency rules. A new RPC offers greater protocol freedom at the cost of another routing and compatibility surface. A session-factory hook can install a registry, but is complementary to the per-request relation contract.

Acceptance should include malformed envelopes rejected before child callbacks, empty and uneven input partitions, schema preservation, and planning without execution. Source fixtures exercise these boundaries; they do not establish sorted or co-partitioned inputs, arbitrary host-service propagation, or indexed spatial-join planning. Those require separate contracts and evidence.

The decision is whether this bounded physical-input interface is sufficient for the first table-producing extensions. Once a relation may change native state, laziness alone is insufficient: [commands and receipts](#) make execution and acknowledgement explicit.

4. Commands and receipts

[Relations](#) describe work without executing it. Some operations, such as staging or dropping a graph, must also change state. The prototype represents them as relations with mutating verbs and small receipt results. The client explicitly collects the receipt. This reuses the relation transport and input handling while making the execution boundary visible to the caller.

The **public behavior** should distinguish planning, attempted execution, committed state and an acknowledged result. Constructing a DataFrame or asking for its schema must not mutate state. A successful stage receipt identifies the graph, node and edge counts, and revision; a drop receipt reports whether removal occurred. If execution or acknowledgement is interrupted, absence of a receipt is not proof that no mutation happened. Callers need an indeterminate outcome, not an automatic retry presented as harmless. None of this promises cross-request exactly-once execution or durable recovery.

The pinned **implementation** uses Nutmeg's `MutationTable` and an execution node with one output partition. Its shared attempt state changes from ready to running before the mutation; completion caches the receipt batch. Another execution of that same completed object can return the cached receipt. A running or previously failed attempt is refused rather than executed again. A newly planned request has a new attempt object, so this mechanism is not a persistent idempotency key. [Mutation state and execution](#). The client's `stage` and `drop` methods collect the result eagerly; ordinary algorithm results remain lazy. [Client behavior](#).

A separate safeguard sits in Sail's scheduler: a task region containing driver-native execution receives one attempt, including native reads. That conservative placement rule prevents the scheduler from silently replaying a region whose effects it cannot classify. It does not prevent a client from submitting a new request, make a lost receipt recoverable, or establish transactions across operations. The package's cached receipt and the scheduler's region policy solve different replay problems.

Spark Connect's raw `Command.extension` is a fair alternative and is not implemented by this prototype. It would distinguish effects at the protocol level, while requiring command dispatch, acknowledgement and input semantics. The relation approach reduces the initial surface but cannot infer that an arbitrary relation is safe to replay. A durable operation identifier and result log would be additional design work under either transport.

Acceptance should verify no mutation during EXPLAIN, one effect on repeated execution of a single provider, and explicit failure after an interrupted attempt; a disconnected client must never imply rollback. The decision is whether receipts plus conservative non-replay suffice initially, or whether read/effect classification is required first. That leads directly to [placement and replay](#).

5. Placement and replay

The [command contract](#) makes an uncertain mutation outcome explicit. Placement determines where that mutation may execute; replay policy determines whether another attempt is safe. These are separate decisions. A function over ordinary input batches can be stateless even when it runs on a worker, while an operation tied to a process-local graph needs that owner's state to remain available.

The public contract should require Sail to respect declared placement, reject invalid or expired ownership, and avoid replaying work whose effects are unknown. A serialized reference must identify the intended owner and plan, and decoding must validate input arity and schemas. A live pointer cannot identify an object in another process. This does not require publishing Sail's registry layout or scheduler types as extension APIs.

At the review commit, `bd8ce9ae`, `DriverExtensionExec` references a bound plan through owner and plan identifiers. Its registry holds weak references so it does not independently prolong query state. A driver-native region receives one scheduler attempt, including reads, because the host has no general classification of extension effects. The host input adapter gathers partitions for this initial driver policy. These are concrete implementation choices, with the ownership and retry checks visible in the [driver boundary](#) and [scheduler](#). One attempt within a job does not provide deduplication across client requests.

The next alternative need not be replicated native state. A stateless foreign worker operator could use serializable parameters and ordinary partitioned inputs, provided its codec, execution context, ownership and replay behavior are defined. Declared replay-safe reads are another possible increment. Recoverable stateful worker stages require a larger design. None follows merely from accepting an execution-plan capsule. Keeping these alternatives separate also preserves Sail's goal of engine-visible distributed execution, illustrated by its [lakehouse planning design](#).

Acceptance should demonstrate rejection of wrong-owner, expired and wrong-schema handles; no scheduler replay after a mutation loses its acknowledgement; and ordinary retry behavior for unaffected regions. Explain output should expose the placement boundary. A later distributed extension must also demonstrate the partitioning properties it advertises, rather than inherit an accidental single-partition assumption.

The immediate decision is whether conservative driver placement and one attempt are acceptable initial policies. Approving them should not make gathering or `DriverExtensionExec` permanent public requirements. Once placement is explicit, the next question is [whose memory budget admits the native work](#).

6. Native memory

The [placement decision](#) identifies the process that owns native work. Memory admission must then account for that work alongside Sail's own operators. An extension that allocates substantial state outside the host's budget can exhaust memory even when every participating DataFusion operator honors its reservation.

The public invariant is that participating allocations have admission before they are made, and retain that admission until their storage and allocation authority are gone. Native state, snapshots, producers and exported Arrow buffers can outlive the initiating request. Returning quota when a session closes is therefore too early if any of those owners survives. The extension must cooperate with this protocol; a lease is not an allocator interception mechanism or a tenant security boundary.

The review implementation prepays the manifest's `memory_bytes` from the host DataFusion pool. A small resource ABI carries a byte limit and opaque ownership callbacks; the extension subdivides the reservation internally. The reservation is non-spillable, including unused capacity. At `bd8ce9ae`, this bridge serves participating **driver-native allocations**: the [manifest validator](#) rejects `memory_bytes` with another placement. It does not supply equivalent admission for worker scalar UDF allocations. The [host reservation](#) accounts participating bytes, not process RSS.

With extensions enabled, the standard session manager shares one admission domain with its sessions and in-process workers. Separately started worker factories own separate domains. This topology and the pool's configured limit must be disclosed; equal numeric limits do not imply shared ownership or a cluster-wide budget. Runtime, Python, transport, metadata and other unaccounted allocations require additional headroom.

Prepayment is a straightforward initial policy, but hierarchical pools or dynamic requests to a common admission coordinator are valid alternatives. The unsafe alternative is two independent pools each promising the same capacity. Dynamic lending could improve utilization, at the cost of defining failure, concurrency and reclamation across the boundary. Native spilling additionally requires an extension-specific recovery path; it is not created by setting a host spill option.

Acceptance should show host operators and native reservations contending against the same finite limit, rejected admission before allocation, and a retained Arrow output preventing early quota reuse. Last-owner tests must inspect storage lifetime as well as reservation counters. Passing a counter-only check cannot prove that the last buffer was already freed.

The decision is whether coarse, prepaid admission is acceptable initially, with its scope and idle-capacity cost explicit. The public ownership invariant should survive a later policy change. [Lifecycle and teardown](#) determine what happens when an admitted owner does not finish promptly.

7. Lifecycle and teardown

The [memory contract](#) ends only when the final owner releases its storage and allocation authority. Cancellation begins that process; it does not prove completion. A dropped client connection, interrupted query or deleted session may still leave native producers or exported buffers alive.

The public contract needs obligations on both sides. Sail must stop accepting new work for a closing session, request cancellation, release its streams and buffers, and preserve an appropriate terminal identity for client reattachment. The extension must cooperate with cancellation, stop creating new work, and retain admission for surviving state and outputs. Neither side may present a cancellation request as evidence that native execution has stopped. A shutdown result should distinguish successful draining from outstanding ownership.

The implementation at `bd8ce9ae` drains protocol executors before discarding session contexts. It separately tracks cleanup for sessions already deleted from the manager and waits for native leases to be released. The [shutdown sequence](#) and [cleanup task set](#) are host machinery, not API types an extension should implement. A Python owner also acquires the GIL for final destruction, avoiding a deferred `decref` that could otherwise retain quota until another Python entry. Some lifecycle corrections affect ordinary Sail execution without the extension flag; their review and regression coverage should state that scope.

The unresolved operational policy is consequential: the native-release wait has no deadline. Waiting indefinitely preserves accounting but can prevent graceful shutdown. A bounded wait that reports failure is viable only if live owners remain charged and the process prevents their state from being reused as though cleanup succeeded. Process termination is another policy, with a larger failure boundary. A timeout alone cannot safely revoke arbitrary native memory or preempt a noncooperative thread. The choice must specify who acts, what remains live, and what the caller observes.

Acceptance should cover cancellation during input and output, deletion followed immediately by shutdown, late plan rejection, and a retained output released after the request ends. A deliberately blocked owner should exercise the chosen timeout or termination policy. Tests should verify both terminal client behavior and continued admission while the owner survives, then final release when it actually exits. Forced termination needs separate process-level evidence.

The review decision is therefore both the cooperative lifecycle contract and the host's failure policy. It cannot be deferred behind a claim that cleanup is already complete. Finally, [compatibility](#) determines which independently built extensions may safely participate in this protocol.

8. Packaging and compatibility

The [lifecycle contract](#) assumes that both sides interpret their shared objects correctly. Packaging makes an extension discoverable; compatibility determines whether Sail may call into it. Those are related concerns, but a wheel that installs successfully is not by itself evidence that its native boundary is compatible.

The public contract should identify the supported extension API and native interchange versions, reject declared incompatibility before accessing capsule layouts, and define package identity consistently across driver and workers. It must also make the trust boundary explicit: native packages promise the named capsule layout and ownership rules. Their declarations are not a proof that arbitrary native code is safe. Function pointers remain process-local; workers resolve their installed implementation from a serialized identity.

At `bd8ce9ae`, the loader accepts `api_version: 1`, DataFusion 55.1.0 and Arrow 59.3.0, checking these declarations before binding native objects. Its [manifest checks](#) are deliberately exact. The [package fingerprint](#) combines manifest options and installed package-file contents. It detects a different installed package identity; it is not a universal ABI certificate, an approved-artifact signature, or a fingerprint of every external dependency. Sail does not distribute extension wheels to workers. Operators must install matching packages there.

This is a qualified prototype boundary, not a promise that extensions must forever share Sail's release cadence. The upstream maintainer's [extension design response](#) explicitly favors Python distribution and DataFusion FFI so Sail and extensions can release independently, while treating session mutators as implementation details. Avoiding a public Sail Rust trait supports that direction. Exact pins currently limit the demonstrated compatibility; they do not establish the range that a future stable interface could support.

A tested compatibility range would reduce rebuilds but needs evidence for every admitted boundary and a policy for breaking changes. Exact artifact pairs are easier to qualify initially and can fail closed when declarations differ. A custom C ABI could provide another stability boundary, but requires its own versioning and ownership design. It is an alternative with maintenance costs, not an impossibility ruled out by choosing wheels.

Acceptance should include wrong API and engine-version declarations, invalid capsule names or types, missing worker packages, and changed package identities. Positive controls should exercise the qualified artifacts across the actual planning, codec and execution path. Only those tested pairs should be advertised as supported; matching version strings alone cannot extend the claim.

The decision is the initial support statement and the evidence required to widen it. With the eight boundaries explicit, the [code listings](#) show the concrete extension surface reviewers can inspect next.

Complete code and reading path

The small author sample

Read the first five listings consecutively: the native adapter, its Python bootstrap, the two package manifests, and the spatial SQL/shuffle client. The first four total 246 lines. The client is copied verbatim from the pinned tutorial; its import of `functions` as `F` is retained even though this example uses SQL strings. Expected assertions are distance 5.0 and distances 0 through 16 after repartition. These are expectations in the existing example, not a new execution result from this documentation build.

Every printed source file below is complete. The client is the complete Python body of the tutorial example; server startup is its complete shell block. Their source line intervals are recorded. Rust, Python, TOML, shell, protobuf and patch listings have language tags for color syntax rendering. Long lines may wrap in exported editions; the source bytes are preserved in the Markdown and code bundle.

Running the sample

Use a dedicated checkout with Rust 1.97.1, shared-library Python 3.12, uv, protoc and a C/C++ toolchain; Sedona also needs GEOS 3.12 or newer. The pinned tutorial describes platform setup. Review these commands before running them: the original build script synchronizes its virtual environment and builds both Sedona and Nutmeg. This document does not run it.

```
1 git clone https://github.com/querygraph/sail.git sail-extension-review
2 cd sail-extension-review
3 git checkout --detach bd8ce9ae8839477e2c08a0475ab7900b115c5366
4 bash examples/extensions/scripts/build.sh
5 .venv/bin/python examples/extensions/sedona/scripts/smoke.py
```

In terminal A use the complete [server startup](#) listing. In terminal B save the [client](#) as a Python file and run it with `SPARK_CONNECT_MODE_ENABLED=1 .venv/bin/python <client-file.py>` from the checkout. Stop the server before changing execution modes. To exercise separate workers, retain the startup exports and change its final launch to `SAIL_MODE=local-cluster SAIL_EXPERIMENTAL_PROCESS_WORKERS=1` followed by the same executable and arguments. A local shuffle alone does not prove remote worker execution; placement must be observed in the selected deployment.

The code bundle preserves the original file paths, complete Cargo lockfile, licenses and notices, and a SHA-256 manifest. Third-party library implementations are pinned dependencies, not reprinted here. The much larger Sail integration is documented separately in the [host implementation companion](#). That companion includes the complete historical host patch, with a comparison against the separately pinned plain-Sail revision.

Listings

The native scalar adapter

Source: [examples/extensions/sedona/src/lib.rs](#). 170 lines.

```
1 /// The extension is an independent dynamic library with no Sail crate dependency.
2
3 use std::panic::{catch_unwind, AssertUnwindSafe};
4 use std::sync::Arc;
5
6 use datafusion_common::config::ConfigOptions;
7 use datafusion_common::{Result, ScalarValue};
8 use datafusion_expr::{ColumnarValue, ScalarUDF, ScalarUDFImpl};
9 use datafusion_ffi::udf::FFI_ScalarUDF;
10 use pyo3::exceptions::PyRuntimeError;
11 use pyo3::prelude::*;
12 use pyo3::types::PyCapsule;
13 use sedona_common::SedonaOptions;
14 use sedona_expr::scalar_udf::{ScalarKernelRef, SedonaScalarKernel};
15 use sedona_schema::dataTypes::SedonaType;
```

```

16
17 /// FFI 55.1 cannot preserve Sedona's typed runtime services in ConfigOptions.
18 /// Each bound session therefore captures an explicit, immutable Sedona default
19 /// snapshot. Session SET propagation is outside this local proof of concept.
20 #[derive(Debug)]
21 struct SnapshotKernel {
22     inner: ScalarKernelRef,
23     options: Arc<ConfigOptions>,
24 }
25
26 fn guarded<T>(call: impl FnOnce() -> Result<T>) -> Result<T> {
27     catch_unwind(AssertUnwindSafe(call))
28     .unwrap_or_else(|_| datafusion_common::exec_err!("Apache SedonaDB native scala
29 r panicked"))
30 }
31
32 impl SedonaScalarKernel for SnapshotKernel {
33     fn return_type(&self, args: &[SedonaType]) -> Result<Option<SedonaType>> {
34         guarded(|| self.inner.return_type(args))
35     }
36
37     fn return_type_from_args_and_scalars(
38         &self,
39         args: &[SedonaType],
40         scalars: &[Option<&ScalarValue>],
41     ) -> Result<Option<SedonaType>> {
42         guarded(|| self.inner.return_type_from_args_and_scalars(args, scalars))
43     }
44
45     fn invoke_batch(
46         &self,
47         arg_types: &[SedonaType],
48         args: &[ColumnarValue],
49     ) -> Result<ColumnarValue> {
50         guarded(|| self.inner.invoke_batch(arg_types, args))
51     }
52
53     fn invoke_batch_from_args(
54         &self,
55         arg_types: &[SedonaType],
56         args: &[ColumnarValue],
57         return_type: &SedonaType,
58         rows: usize,
59         host_options: Option<&ConfigOptions>,
60     ) -> Result<ColumnarValue> {
61         guarded(|| {
62             self.inner.invoke_batch_from_args(
63                 arg_types,
64                 args,
65                 return_type,
66                 rows,
67                 Some(&self.options),
68             )
69         })
70     }
71 }
72
73 #[pyclass(skip_from_py_object)]
74 #[derive(Clone)]
75 struct NativeScalarUdf {
76     inner: Arc<ScalarUDF>,
77 }
78
79 #[pymethods]
80 impl NativeScalarUdf {
81     fn name(&self) -> &str {
82         self.inner.name()
83     }
84
85     fn aliases(&self) -> Vec<String> {
86         self.inner.aliases().to_vec()
87     }
88
89     fn __datafusion_scalar_udf__<'py>(&self, py: Python<'py>) -> PyResult<Bound<'py, P
90 yCapsule>> {
91         PyCapsule::new_with_value(

```

```

90         py,
91         FFI_ScalarUDF::from(Arc::clone(&self.inner)),
92         c"datafusion_scalar_udf",
93     )
94 }
95 }
96
97 #[pyclass]
98 struct BoundSedona {
99     #[pyo3(get)]
100     session_id: String,
101     functions: Vec<NativeScalarUdf>,
102 }
103
104 #[pymethods]
105 impl BoundSedona {
106     #[new]
107     fn new(session_id: String) -> PyResult<Self> {
108         let mut set = sedona_functions::register::default_function_set();
109         for (name, kernels) in sedona_geos::register::scalar_kernels() {
110             set.add_scalar_udf_impl(name, kernels)
111                 .map_err(|e| PyRuntimeError::new_err(e.to_string()))?;
112         }
113         let mut options = ConfigOptions::new();
114         options.extensions.insert(SedonaOptions::default());
115         let options = Arc::new(options);
116         let mut functions: Vec<_> = set
117             .scalar_udfs()
118             // The first host policy rejects built-in shadowing. Omit the
119             // complete UDF (including aliases) when any name would collide.
120             .filter(|function| {
121                 const BUILTINS: &[&str] = &[
122                     "st_asbinary",
123                     "st_geomfromwkb",
124                     "st_geogfromwkb",
125                     "st_setsrid",
126                     "st_srid",
127                 ];
128                 !BUILTINS.contains(&function.name())
129                     && !function
130                         .aliases()
131                         .iter()
132                         .any(|name| BUILTINS.contains(&name.as_str()))
133             })
134             .map(|function| {
135                 let kernels = function
136                     .kernels()
137                     .iter()
138                     .map(|kernel| {
139                         Arc::new(SnapshotKernel {
140                             inner: Arc::clone(kernel),
141                             options: Arc::clone(&options),
142                         }) as ScalarKernelRef
143                     })
144                     .collect();
145                 NativeScalarUdf {
146                     inner: Arc::new(function.clone().with_kernels(kernels).into()),
147                 }
148             })
149             .collect();
150         functions.sort_by(|a, b| a.inner.name().cmp(b.inner.name()));
151         Ok(Self {
152             session_id,
153             functions,
154         })
155     }
156
157     fn scalar_udfs(&self) -> Vec<NativeScalarUdf> {
158         self.functions.clone()
159     }
160 }
161
162 #[pymodule]
163 fn _native(module: &Bound<'_, PyModule>) -> PyResult<()> {
164     module.add_class::<BoundSedona>()?;
165     module.add_class::<NativeScalarUdf>()?;
166     Ok(())

```

```

167 }
168
169 #[cfg(test)]
170 mod tests;

```

Discovery and binding

Source: [examples/extensions/sedona/python/sail_sedona/__init__.py](#). 23 lines.

```

1  """Apache SedonaDB functions exported through the DataFusion 55.1 capsule API."""
2
3
4  class SedonaExtension:
5      def manifest(self):
6          # Keep validation possible before importing any native capsule provider.
7          return {
8              "name": "sedona",
9              "version": "0.1.0",
10             "api_version": 1,
11             "datafusion_version": "55.1.0",
12             "arrow_version": "59.3.0",
13             "placement": "any",
14             "relation_types": [],
15         }
16
17     def bind(self, session_id):
18         from ._native import BoundSedona
19
20         return BoundSedona(session_id)
21
22
23 extension = SedonaExtension()

```

Native package dependencies

Source: [examples/extensions/sedona/Cargo.toml](#). 34 lines.

```

1  [package]
2  name = "sail-sedona-extension"
3  version = "0.1.0"
4  edition = "2021"
5  publish = false
6  license = "Apache-2.0"
7
8  # This wheel is deliberately compiled independently of Sail.
9  [workspace]
10 exclude = [".deps/sedona-db", ".deps/sedona-db/*"]
11
12 [lib]
13 name = "_native"
14 crate-type = ["cdylib"]
15
16 [dependencies]
17 arrow-schema = "=59.3.0"
18 datafusion-common = { version = "=55.1.0", default-features = false }
19 datafusion-expr = { version = "=55.1.0", default-features = false }
20 datafusion-ffi = "=55.1.0"
21 pyo3 = "=0.29.0"
22 sedona-common = { path = ".deps/sedona-db/rust/sedona-common" }
23 sedona-expr = { path = ".deps/sedona-db/rust/sedona-expr" }
24 sedona-functions = { path = ".deps/sedona-db/rust/sedona-functions" }
25 sedona-geos = { path = ".deps/sedona-db/c/sedona-geos" }
26 sedona-schema = { path = ".deps/sedona-db/rust/sedona-schema" }
27
28 [profile.dev]
29 debug = 0
30 incremental = false
31
32 [profile.release]
33 debug = 0
34 incremental = false

```

Python packaging and entry point

Source: [examples/extensions/sedona/pyproject.toml](#). 19 lines.

```
1 [build-system]
2 requires = ["maturin>=1.8,<2"]
3 build-backend = "maturin"
4
5 [project]
6 name = "sail-sedona-extension"
7 version = "0.1.0"
8 description = "Apache SedonaDB native scalar extension for the Sail extension proof of
  concept"
9 requires-python = ">=3.12"
10 license = "Apache-2.0"
11 license-files = ["LICENSE-SEDONADB", "NOTICE-SEDONADB", "LICENSE-GEOS", "NOTICE"]
12
13 [project.entry-points."pysail.extensions"]
14 sedona = "sail_sedona:extension"
15
16 [tool.maturin]
17 features = ["pyo3/extension-module"]
18 module-name = "sail_sedona._native"
19 python-source = "python"
```

Complete spatial SQL and shuffle client

Source: [examples/extensions/TUTORIAL.md](#). 20 lines.

```
1 from pyspark.sql.connect.session import SparkSession
2 from pyspark.sql import functions as F
3 spark = SparkSession.builder.remote("sc://127.0.0.1:50051").create()
4 try:
5     row = spark.sql("""SELECT
6         ST_AsText(ST_Point(1.0, 2.0)) AS wkt,
7         ST_Distance(ST_Point(0.0, 0.0), ST_Point(3.0, 4.0)) AS distance
8     """).first()
9     print(row)
10    assert row.distance == 5.0
11    points = spark.range(0, 17, numPartitions=4).selectExpr(
12        "id", "ST_Point(CAST(id AS DOUBLE), 2.0) AS geom")
13    rows = points.repartition(4, "id").selectExpr(
14        "id", "ST_AsText(geom) AS wkt",
15        "ST_Distance(geom, ST_Point(0.0, 2.0)) AS distance"
16    ).orderBy("id").collect()
17    assert [r.distance for r in rows] == [float(i) for i in range(17)]
18    print("Sedona: 17 geometry rows survived the shuffle")
19 finally:
20    spark.stop()
```

Complete native adapter tests

Source: [examples/extensions/sedona/src/tests.rs](#). 178 lines.

```
1 use arrow_schema::{Field, FieldRef};
2 use datafusion_expr::{ReturnFieldArgs, ScalarFunctionArgs, ScalarUDFImpl};
3 use datafusion_ffi::udf::ForeignScalarUDF;
4
5 use super::*;
6
7 #[test]
8 fn kernel_panic_becomes_a_result_before_the_ffi_boundary() {
9     let result: Result<> = guarded(|| panic!("injected native kernel failure"));
10     assert!(result
11         .unwrap_err()
12         .to_string()
13         .contains("native scalar panicked"));
14 }
15
16 fn argument(value: ScalarValue) -> (ColumnarValue, FieldRef) {
17     let field = Arc::new(Field::new("arg", value.data_type(), true));
18     (ColumnarValue::Scalar(value), field)
19 }
20
```

```

21 fn one_value(value: ColumnarValue) -> ScalarValue {
22   // The Arrow FFI materializes scalar arguments as one-row arrays. Assert
23   // cardinality explicitly, then compare the exact value rather than the
24   // implementation's Scalar/Array representation.
25   let array = value.into_array(1).unwrap();
26   assert_eq!(array.len(), 1);
27   ScalarValue::try_from_array(&array, 0).unwrap()
28 }
29
30 fn call(
31   bound: &BoundSedona,
32   name: &str,
33   args: Vec<(ColumnarValue, FieldRef)>,
34 ) -> Result<(ColumnarValue, FieldRef)> {
35   let function = bound
36     .functions
37     .iter()
38     .find(|f| f.inner.name() == name)
39     .unwrap();
40   // A foreign marker prevents the same-library optimization bypassing FFI.
41   extern "C" fn foreign_marker() -> usize {
42     0
43   }
44   let mut ffi = FFI_ScalarUDF::from(Arc::clone(&function.inner));
45   ffi.library_marker_id = foreign_marker;
46   let imported: Arc<dyn ScalarUDFImpl> = ffi.into();
47   assert!(imported.is::<ForeignScalarUDF>());
48   let function = ScalarUDF::new_from_shared_impl(imported);
49   let (values, fields): (Vec<_>, Vec<_>) = args.into_iter().unzip();
50   let rows = values
51     .iter()
52     .filter_map(|value| match value {
53       ColumnarValue::Array(array) => Some(array.len()),
54       ColumnarValue::Scalar(_) => None,
55     })
56     .next()
57     .unwrap_or(1);
58   let scalars = values
59     .iter()
60     .map(|v| match v {
61       ColumnarValue::Scalar(s) => Some(s),
62       ColumnarValue::Array(_) => None,
63     })
64     .collect::<Vec<_>>();
65   let result_field = function.return_field_from_args(ReturnFieldArgs {
66     arg_fields: &fields,
67     scalar_arguments: &scalars,
68   })?;
69   let value = function.invoke_with_args(ScalarFunctionArgs {
70     args: values,
71     arg_fields: fields,
72     number_rows: rows,
73     return_field: Arc::clone(&result_field),
74     config_options: Arc::new(ConfigOptions::new()),
75   })?;
76   Ok((value, result_field))
77 }
78
79 #[test]
80 fn ffi_geometry_metadata_and_geos_predicate() {
81   let bound = BoundSedona::new("ffi-test".to_string()).unwrap();
82   let point = call(
83     &bound,
84     "st_point",
85     vec![
86       argument(ScalarValue::Float64(Some(1.0))),
87       argument(ScalarValue::Float64(Some(2.0))),
88     ],
89   )
90   .unwrap();
91   let polygon = call(
92     &bound,
93     "st_geomfromwkt",
94     vec![argument(ScalarValue::Utf8(Some(
95       "POLYGON ((0 0, 3 0, 3 3, 0 3, 0 0))".to_string(),
96     )))],
97   )

```

```

98     .unwrap();
99     let result = call(&bound, "st_intersects", vec![point.clone(), polygon]).unwrap();
100    assert_eq!(one_value(result.0), ScalarValue::Boolean(Some(true)));
101    let result = call(&bound, "st_astext", vec![point]).unwrap();
102    assert_eq!(
103        one_value(result.0),
104        ScalarValue::Utf8(Some("POINT(1 2)".into()))
105    );
106 }
107
108 #[test]
109 fn ffi_null_geometry_and_aliases_are_preserved() {
110     let bound = BoundSedona::new("null-test".to_string()).unwrap();
111     let null = call(
112         &bound,
113         "st_geomfromwkt",
114         vec![argument(ScalarValue::Utf8(None))],
115     )
116     .unwrap();
117     let result = call(&bound, "st_astext", vec![null]).unwrap();
118     assert_eq!(one_value(result.0), ScalarValue::Utf8(None));
119     let from_wkt = bound
120         .functions
121         .iter()
122         .find(|f| f.inner.name() == "st_geomfromwkt")
123         .unwrap();
124     assert!(from_wkt.aliases().iter().any(|a| a == "st_geomfromtext"));
125 }
126
127 #[test]
128 fn envelope_bypass_state_matches_partial_update_for_nulls_and_filters() {
129     use datafusion_common::arrow::array::{BooleanArray, StringArray};
130     use datafusion_expr::EmitTo;
131
132     let bound = BoundSedona::new("aggregate-port".to_string()).unwrap();
133     let texts = ColumnarValue::Array(Arc::new(StringArray::from(vec![
134         Some("POINT(1 2)"),
135         None,
136         Some("POINT(10 20)"),
137         Some("POINT(3 4)"),
138     ])));
139     let geometry = call(
140         &bound,
141         "st_geomfromwkt",
142         vec![(
143             texts,
144             Arc::new(Field::new("wkt", arrow_schema::DataType::Utf8, true)),
145         )],
146     )
147     .unwrap();
148     let input_type = SedonaType::from_storage_field(&geometry.1).unwrap();
149     let args = [input_type];
150     let functions = sedona_functions::register::default_function_set();
151     let udf = functions.aggregate_udf("st_envelope_agg").unwrap();
152     let (kernel, output_type) = udf
153         .kernels()
154         .iter()
155         .find_map(|kernel| {
156             kernel
157                 .return_type(&args)
158                 .unwrap()
159                 .map(|output| (kernel, output))
160         })
161     .unwrap();
162     let arrays = [geometry.0.into_array(4).unwrap()];
163     let groups = [0, 0, 1, 1];
164     let filter = BooleanArray::from(vec![Some(true), Some(true), Some(false), Some(true)]);
165     let mut partial = kernel.groups_accumulator(&args, &output_type).unwrap();
166     partial
167         .update_batch(&arrays, &groups, Some(&filter), 3)
168         .unwrap();
169     let expected = partial.evaluate(EmitTo::All).unwrap();
170
171     let bypass = kernel.groups_accumulator(&args, &output_type).unwrap();
172     let states = bypass.convert_to_state(&arrays, Some(&filter)).unwrap();
173     let mut merged = kernel.groups_accumulator(&args, &output_type).unwrap();

```

```

174 merged.merge_batch(&states, &groups, 3).unwrap();
175 let actual = merged.evaluate(EmitTo::All).unwrap();
176 assert_eq!(actual.len(), 3);
177 assert_eq!(actual.to_data(), expected.to_data());
178 }

```

Installed-wheel smoke check

Source: `examples/extensions/sedona/scripts/smoke.py`. 31 lines.

```

1  """Verify the installed wheel manifest and every native capsule without Sail."""
2
3  import ctypes
4  from importlib.metadata import entry_points
5
6  from sail_sedona import extension
7
8  manifest = extension.manifest()
9  assert manifest == {
10     "name": "sedona",
11     "version": "0.1.0",
12     "api_version": 1,
13     "datafusion_version": "55.1.0",
14     "arrow_version": "59.3.0",
15     "placement": "any",
16     "relation_types": [],
17 }
18 entries = [e for e in entry_points(group="pysail.extensions") if e.name == "sedona"]
19 assert len(entries) == 1
20 assert entries[0].load().manifest() == manifest
21 bound = extension.bind("sedona-wheel-smoke")
22 functions = {f.name(): f for f in bound.scalar_udfs()}
23 assert len(functions) == 128
24 assert {"st_point", "st_geomfromwkt", "st_astext", "st_intersects", "st_distance"} <=
    functions.keys()
25 assert "st_geomfromtext" in functions["st_geomfromwkt"].aliases()
26 assert not {"st_asbinary", "st_geomfromwkb", "st_geogfromwkb", "st_setsrid", "st_srid"}
    & functions.keys()
27 is_valid = ctypes.pythonapi.PyCapsule_IsValid
28 is_valid.argtypes = [ctypes.py_object, ctypes.c_char_p]
29 is_valid.restype = ctypes.c_int
30 assert all(is_valid(f.__datafusion_scalar_udf__(), b"datafusion_scalar_udf") for f in
    functions.values())
31 print({"manifest": manifest, "scalars": len(functions), "capsules": "valid"})

```

Pinned dependency preparation

Source: `examples/extensions/sedona/scripts/prepare.py`. 57 lines.

```

1  #!/usr/bin/env python3
2  """Fetch a fixed Apache SedonaDB revision and apply the reviewed DF55 port."""
3
4  from pathlib import Path
5  import os
6  import subprocess
7  import tempfile
8
9  ROOT = Path(__file__).resolve().parents[1]
10 SOURCE = ROOT / ".deps" / "sedona-db"
11 REVISION = "0a1993d9be8bcf52150593ad08fc6a3412d50f29"
12 PATCH = ROOT / "patches" / "datafusion-55.patch"
13
14
15 def git(*args):
16     return subprocess.run(["git", "-C", str(SOURCE), *args], check=True)
17
18
19 if not SOURCE.exists():
20     SOURCE.parent.mkdir(parents=True, exist_ok=True)
21     subprocess.run(
22         ["git", "clone", "--filter=blob:none", "--no-checkout",
23          "https://github.com/apache/sedona-db.git", str(SOURCE)],

```

```

24         check=True,
25     )
26     git("checkout", "--detach", REVISION)
27 else:
28     actual = subprocess.check_output(
29         ["git", "-C", str(SOURCE), "rev-parse", "HEAD"], text=True
30     ).strip()
31     if actual != REVISION:
32         raise SystemExit(f"Expected Apache SedonaDB {REVISION}, found {actual}")
33
34     applied = subprocess.run(
35         ["git", "-C", str(SOURCE), "apply", "--reverse", "--check", str(PATCH)],
36         capture_output=True,
37     ).returncode == 0
38     if not applied:
39         git("apply", "--check", str(PATCH))
40         git("apply", str(PATCH))
41
42     # A reversible patch alone does not prove that unrelated source was unchanged.
43     # Construct the expected patched tree with an isolated index, without resetting
44     # the checkout or touching its own index, and compare every tracked source file.
45     with tempfile.TemporaryDirectory(prefix="sail-sedona-source-") as temporary:
46         env = dict(os.environ, GIT_INDEX_FILE=str(Path(temporary) / "index"))
47         command = ["git", "-C", str(SOURCE)]
48         subprocess.run(command + ["read-tree", REVISION], env=env, check=True)
49         subprocess.run(command + ["apply", "--cached", str(PATCH)], env=env, check=True)
50         expected = subprocess.check_output(command + ["write-tree"], env=env, text=True).s
51         trip()
52         subprocess.run(command + ["diff", "--exit-code", expected, "--"], check=True)
53         extras = subprocess.check_output(
54             command + ["ls-files", "--others", "--exclude-standard"], text=True
55         ).strip()
56         if extras:
57             raise SystemExit(f"Unexpected untracked files in pinned SedonaDB source:\n{ext
ras}")
58     print(f"Apache SedonaDB {REVISION}, DF55 patch ready at {SOURCE}")

```

Complete SedonaDB compatibility patch

Source: [examples/extensions/sedona/patches/datafusion-55.patch](#). 111 lines.

```

1 diff --git a/Cargo.toml b/Cargo.toml
2 index 4f7c991..4d01e6a 100644
3 --- a/Cargo.toml
4 +++ b/Cargo.toml
5 @@ -71,14 +71,14 @@ categories = ["science::geo", "database"]
6   adbc_core = ">=0.24.0"
7   adbc_ffi = ">=0.24.0"
8   approx = "0.5"
9   -arrow = { version = "58.3.0", features = ["prettyprint", "ffi", "chrono-tz"] }
10  -arrow-array = { version = "58.3.0" }
11  -arrow-buffer = { version = "58.3.0" }
12  -arrow-cast = { version = "58.3.0" }
13  -arrow-data = { version = "58.3.0" }
14  -arrow-ipc = { version = "58.3.0" }
15  -arrow-json = { version = "58.3.0" }
16  -arrow-schema = { version = "58.3.0" }
17  +arrow = { version = "=59.3.0", features = ["prettyprint", "ffi", "chrono-tz"] }
18  +arrow-array = { version = "=59.3.0" }
19  +arrow-buffer = { version = "=59.3.0" }
20  +arrow-cast = { version = "=59.3.0" }
21  +arrow-data = { version = "=59.3.0" }
22  +arrow-ipc = { version = "=59.3.0" }
23  +arrow-json = { version = "=59.3.0" }
24  +arrow-schema = { version = "=59.3.0" }
25  async-trait = { version = "0.1.87" }
26  bytemuck = "1.25"
27  byteorder = "1"
28  @@ -86,21 +86,21 @@ bytes = "1.11"
29   chrono = { version = "0.4.41", default-features = false }
30   comfy-table = { version = "8.0" }
31   criterion = { version = "0.8", features = ["html_reports"] }
32  -datafusion = { version = "54.1.0", default-features = false }
33  -datafusion-catalog = { version = "54.1.0" }
34  -datafusion-common = { version = "54.1.0", default-features = false }

```

```

35 -datafusion-common-runtime = { version = "54.1.0", default-features = false }
36 -datafusion-proto = { version = "54.1.0", default-features = false }
37 -datafusion-datasource = { version = "54.1.0", default-features = false }
38 -datafusion-datasource-parquet = { version = "54.1.0" }
39 -datafusion-execution = { version = "54.1.0", default-features = false }
40 -datafusion-expr = { version = "54.1.0", default-features = false }
41 -datafusion-ffi = { version = "54.1.0" }
42 -datafusion-optimizer = { version = "54.1.0" }
43 -datafusion-physical-expr = { version = "54.1.0" }
44 -datafusion-physical-plan = { version = "54.1.0" }
45 -datafusion-pruning = { version = "54.1.0" }
46 -datafusion-session = { version = "54.1.0" }
47 +datafusion = { version = "55.1.0", default-features = false }
48 +datafusion-catalog = { version = "55.1.0" }
49 +datafusion-common = { version = "55.1.0", default-features = false }
50 +datafusion-common-runtime = { version = "55.1.0", default-features = false }
51 +datafusion-proto = { version = "55.1.0", default-features = false }
52 +datafusion-datasource = { version = "55.1.0", default-features = false }
53 +datafusion-datasource-parquet = { version = "55.1.0" }
54 +datafusion-execution = { version = "55.1.0", default-features = false }
55 +datafusion-expr = { version = "55.1.0", default-features = false }
56 +datafusion-ffi = { version = "55.1.0" }
57 +datafusion-optimizer = { version = "55.1.0" }
58 +datafusion-physical-expr = { version = "55.1.0" }
59 +datafusion-physical-plan = { version = "55.1.0" }
60 +datafusion-pruning = { version = "55.1.0" }
61 +datafusion-session = { version = "55.1.0" }
62 dirs = "7.0.0"
63 env_logger = "0.11"
64 fastrand = "2.4"
65 @@ -122,8 +122,8 @@ num-traits = { version = "0.2", default-features = false, features
= ["libm"] }
66 object_store = { version = "0.13.2", default-features = false }
67 once_cell = "1.20"
68 parking_lot = "0.12"
69 -parquet = { version = "58.3.0", default-features = false, features = ["arrow", "async
", "geospatial", "object_store"] }
70 -parquet-geospatial = { version = "58.3.0" }
71 +parquet = { version = "59.3.0", default-features = false, features = ["arrow", "asyn
c", "geospatial", "object_store"] }
72 +parquet-geospatial = { version = "59.3.0" }
73 pin-project-lite = "0.2"
74 prost = "0.14.1"
75 pyo3 = { version = "0.29.0" }
76 diff --git a/rust/sedona-functions/src/st_envelope_agg.rs b/rust/sedona-functions/src/
st_envelope_agg.rs
77 index 1b5f9ec..b4bc203 100644
78 --- a/rust/sedona-functions/src/st_envelope_agg.rs
79 +++ b/rust/sedona-functions/src/st_envelope_agg.rs
80 @@ -442,10 +442,29 @@ impl<T: WkbBounder2D + Default + 'static> GroupsAccumulator for
BoundsGroupsAccu
81     &mut self,
82     values: &[ArrayRef],
83     group_indices: &[usize],
84 -     opt_filter: Option<&arrow_array::BooleanArray>,
85     total_num_groups: usize,
86 ) -> Result<()> {
87 -     self.merge_state(values, group_indices, opt_filter, total_num_groups)
88 +     self.merge_state(values, group_indices, None, total_num_groups)
89 + }
90 +
91 + fn convert_to_state(
92 +     &self,
93 +     values: &[ArrayRef],
94 +     opt_filter: Option<&BooleanArray>,
95 + ) -> Result<Vec<ArrayRef>> {
96 +     // The bypass path represents every input row as its own group, while
97 +     // preserving the normal update path's null and FILTER semantics.
98 +     let rows = values[0].len();
99 +     let groups: Vec<_> = (0..rows).collect();
100 +     let mut accumulator = Self::new(self.input_type.clone());
101 +     accumulator.execute_update(
102 +         values,
103 +         &groups,
104 +         opt_filter,
105 +         rows,
106 +         self.input_type.clone(),
107 +     );

```

```

108 +         accumulator.emit_state(EmitTo::All)
109     }
110
111     fn evaluate(&mut self, emit_to: EmitTo) -> Result<ArrayRef> {

```

Original two-wheel build script

Source: [examples/extensions/scripts/build.sh](#). 55 lines.

```

1  #!/usr/bin/env bash
2  # Reproducible local PoC: one host executable, two independent native wheels.
3  set -euo pipefail
4  repo=$(cd "$(dirname "$0")/../../.." && pwd)
5  base="$repo/examples/extensions"
6  venv=${SAIL_EXTENSION_VENV:-"$repo/.venv"}
7  target=${SAIL_EXTENSION_TARGET:-"$repo/target/extensions-poc"}
8  python=${SAIL_EXTENSION_PYTHON:-python3.12}
9  export CARGO_INCREMENTAL=0 CARGO_PROFILE_DEV_DEBUG=0
10 export CARGO_BUILD_JOBS=${CARGO_BUILD_JOBS:-4}
11 df -h "$repo"
12 if [[ ! -x "$venv/bin/python" ]]; then
13     uv venv --python "$python" "$venv"
14 fi
15 uv pip sync --python "$venv/bin/python" "$base/requirements.lock"
16 export PYO3_PYTHON="$venv/bin/python"
17 if [[ "$(uname -s)" == Darwin ]]; then
18     export DYLD_LIBRARY_PATH=$(("$venv/bin/python" -c 'import sysconfig; print(sysconfig.get_config_var("LIBDIR"))')')
19 else
20     export LD_LIBRARY_PATH=$(("$venv/bin/python" -c 'import sysconfig; print(sysconfig.get_config_var("LIBDIR"))')')
21 fi
22 "$venv/bin/python" "$base/sedona/scripts/prepare.py"
23 mkdir -p "$target/wheels"
24 for package in sedona nutmeg; do
25     raw=$(mktemp -d "$target/raw-wheel-$package.XXXXXX")
26     CARGO_TARGET_DIR="$target/$package" "$venv/bin/python" -m maturin build \
27         --manifest-path "$base/$package/Cargo.toml" --locked --profile dev \
28         --interpreter "$venv/bin/python" --out "$raw" --auditwheel repair
29     if [[ "$(uname -s)" == Darwin ]]; then
30         "$venv/bin/python" -m delocate.cmd.delocate_wheel -v -w "$raw/repaired" "$raw"/*.whl
31         raw="$raw/repaired"
32     fi
33     for wheel in "$raw"/*.whl; do
34         if [[ "$package" == sedona ]]; then
35             "$venv/bin/python" "$base/scripts/check_wheel.py" "$wheel" \
36                 --output "$target/sedona-native-dependencies.json"
37         fi
38         # A repaired macOS wheel may acquire a newer minimum OS tag. Remove
39         # stale variants of this package so pip cannot pick the unrepaired one.
40         "$venv/bin/python" - "$wheel" "$target/wheels" <<'PY'
41 from pathlib import Path
42 import shutil
43 import sys
44 wheel, output = map(Path, sys.argv[1:])
45 for old in output.glob(wheel.name.split('-')[0] + '-*.whl'):
46     old.unlink()
47 shutil.copy2(wheel, output / wheel.name)
48 PY
49     done
50 done
51 uv pip install --python "$venv/bin/python" --reinstall "$target/wheels/*.whl"
52 # Pecan keeps its established source directory; its distribution is pyspark-pecan.
53 uv pip install --python "$venv/bin/python" --no-deps "$base/graph-algorithms"
54 CARGO_TARGET_DIR="$target/host" cargo build --manifest-path "$repo/Cargo.toml" --locked -p sail-cli
55 printf 'Host: %s\nPython: %s\nWheels: %s\n' "$target/host/debug/sail" "$venv/bin/python" "$target/wheels"

```

Native wheel dependency check

Source: [examples/extensions/scripts/check_wheel.py](#). 65 lines.

```

1 #!/usr/bin/env python3
2 """Require the Sedona wheel's GEOS dependencies to resolve inside the wheel."""
3 import argparse
4 import json
5 from pathlib import Path
6 import platform
7 import subprocess
8 import tempfile
9 import zipfile
10
11
12 def check(wheel):
13     with tempfile.TemporaryDirectory(prefix="sail-wheel-check-") as temporary:
14         root = Path(temporary).resolve()
15         with zipfile.ZipFile(wheel) as archive:
16             archive.extractall(root)
17             libraries = [path for path in root.rglob("*") if path.is_file() and
18                         (path.name.endswith((".so", ".dylib")) or ".so." in path.name)]
19             geos = [path for path in libraries if "geos" in path.name.lower()]
20             if len(geos) < 2:
21                 raise RuntimeError("Sedona wheel must contain both GEOS C and C++ shared l
22                                     ibraries")
23             links = {}
24             for library in libraries:
25                 if platform.system() == "Darwin":
26                     output = subprocess.check_output(["otool", "-L", str(library)], text=T
27                                                         rue)
28                     dependencies = [line.strip().split(" ", 1)[0] for line in output.spli
29                                     tlines()[1:]]
30                     # A dylib's own install ID appears in -L output but is not a dependenc
31                     y.
32                     ids = subprocess.check_output(["otool", "-D", str(library)], text=Tru
33                                                         e).splitlines()[1:]
34                     dependencies = [name for name in dependencies if name not in ids]
35                     for name in dependencies:
36                         if name.startswith("/usr/lib/", "/System/Library/"):
37                             continue
38                         if not name.startswith("@loader_path/"):
39                             raise RuntimeError(f"external native dependency in {library.na
40                                                         me}: {name}")
41                         resolved = (library.parent / name.removeprefix("@loader_path/")).r
42                                     esolve()
43                         if not resolved.is_relative_to(root) or not resolved.is_file():
44                             raise RuntimeError(f"unresolved bundled dependency: {name}")
45                     elif platform.system() == "Linux":
46                         output = subprocess.check_output(["patchelf", "--print-needed", str(li
47                                                         brary)], text=True)
48                         dependencies = output.splitlines()
49                         bundled = {path.name for path in libraries}
50                         for name in dependencies:
51                             if "geos" in name.lower() and name not in bundled:
52                                 raise RuntimeError(f"external GEOS dependency in {library.nam
53                                                         e}: {name}")
54                             if library.name.startswith("_native"):
55                                 rpath = subprocess.check_output(["patchelf", "--print-rpath", str(
56                                                         library)], text=True)
57                                 if "$ORIGIN" not in rpath:
58                                     raise RuntimeError("native module lacks a wheel-relative libra
59                                                         ry search path")
60                                 else:
61                                     raise RuntimeError("wheel dependency check supports macOS and Linux")
62                             links[str(library.relative_to(root))] = dependencies
63             return {"wheel": wheel.name, "bundled_geos": [str(path.relative_to(root)) for
64                                                         path in geos],
65                     "dependencies": links}
66
67 if __name__ == "__main__":
68     parser = argparse.ArgumentParser(description=__doc__)
69     parser.add_argument("wheel", type=Path)
70     parser.add_argument("--output", type=Path)
71     args = parser.parse_args()
72     result = json.dumps(check(args.wheel), indent=2) + "\n"
73     if args.output:
74         args.output.parent.mkdir(parents=True, exist_ok=True)
75         args.output.write_text(result)

```

```
65     print(result, end="")
```

Python dependency lock

Source: [examples/extensions/requirements.lock](#). 69 lines.

```
1 # This file was autogenerated by uv via the following command:
2 #   uv pip compile --universal --python .venv/bin/python examples/extensions/requirements.in -o examples/extensions/requirements.lock
3 altgraph==0.17.5 ; sys_platform == 'darwin'
4     # via macholib
5 apache-sedona==1.8.0
6     # via -r examples/extensions/requirements.in
7 attrs==26.1.0
8     # via apache-sedona
9 colorama==0.4.6 ; sys_platform == 'win32'
10    # via pytest
11 delocate==0.13.0 ; sys_platform == 'darwin'
12    # via -r examples/extensions/requirements.in
13 googleapis-common-protos==1.75.4
14    # via
15    #   grpcio-status
16    #   pyspark
17 grpcio==1.84.0
18    # via
19    #   grpcio-status
20    #   pyspark
21 grpcio-status==1.84.0
22    # via pyspark
23 iniconfig==2.3.0
24    # via pytest
25 macholib==1.16.4 ; sys_platform == 'darwin'
26    # via delocate
27 maturin==1.9.6
28    # via -r examples/extensions/requirements.in
29 numpy==2.5.3
30    # via
31    #   pandas
32    #   pyspark
33    #   shapely
34 packaging==26.3
35    # via
36    #   delocate
37    #   pytest
38 pandas==3.0.6
39    # via pyspark
40 pluggy==1.6.0
41    # via pytest
42 protobuf==7.36.2
43    # via
44    #   googleapis-common-protos
45    #   grpcio-status
46 py4j==0.10.9.9
47    # via pyspark
48 pyarrow==21.0.0
49    # via
50    #   -r examples/extensions/requirements.in
51    #   pyspark
52 pygments==2.21.0
53    # via pytest
54 pyspark==4.0.1
55    # via -r examples/extensions/requirements.in
56 pytest==8.4.2
57    # via -r examples/extensions/requirements.in
58 python-dateutil==2.9.0.post0
59    # via pandas
60 shapely==2.1.2
61    # via apache-sedona
62 six==1.17.0
63    # via python-dateutil
64 typing-extensions==4.16.0
65    # via
66    #   delocate
67    #   grpcio
68 tzdata==2026.4 ; sys_platform == 'emscripten' or sys_platform == 'win32'
69    # via pandas
```

Complete relation envelope

Source: [crates/sail-spark-connect/proto/sail/extension/v1/extension.proto](#). 20 lines.

```
1 // Experimental local-mode protocol. This schema is a wire contract, not a native ABI.
2 syntax = "proto3";
3 package sail.extension.v1;
4
5 import "spark/connect/base.proto";
6 import "spark/connect/expressions.proto";
7
8 // Pack into Relation.extension with type URL:
9 // type.googleapis.com/sail.extension.v1.SailExtensionRequest
10 message SailExtensionRequest {
11     string payload_type_url = 1;
12     bytes payload = 2;
13     // Only Plan.root is accepted. Names are restored to the input DataFrame's
14     // user-facing names before the native handler receives a physical plan.
15     repeated spark.connect.Plan inputs = 3;
16     // Reserved by the proposal; this PoC rejects any occurrence of this field.
17     repeated spark.connect.Expression input_expressions = 4;
18     // Required value: 1. An omitted proto3 value (0) is rejected.
19     uint32 envelope_version = 5;
20 }
```

Complete optional resource ABI

Source: [crates/sail-native-resource-ffi/src/lib.rs](#). 160 lines.

```
1 /// The small C ABI shared by independently compiled native extensions and Sail.
2 /// An opaque, prepaid memory lease crosses the boundary; Rust ownership and
3 /// allocator layouts stay entirely inside the library that issued the lease.
4 use std::ffi::{CStr, c_void};
5 use std::fmt::{Debug, Formatter};
6 use std::ptr::NonNull;
7 use std::sync::Arc;
8
9 pub const MEMORY_LEASE_CAPSULE: &CStr = c"sail_native_memory_lease_v1";
10
11 #[repr(C)]
12 struct Header {
13     version: u32,
14     size: u32,
15 }
16
17 /// A non-spillable host admission retained until the final clone is released.
18 ///
19 /// Callbacks must be thread-safe and must never unwind. Every instance owns one
20 /// reference. A consumer may inspect the header before touching versioned fields.
21 #[repr(C)]
22 pub struct MemoryLease {
23     header: Header,
24     bytes: u64,
25     opaque: *const c_void,
26     retain: unsafe extern "C" fn(*const c_void),
27     release: unsafe extern "C" fn(*const c_void),
28 }
29
30 /// SAFETY: constructors require a Send + Sync owner; imports promise the same
31 /// thread-safe callback contract. The opaque object is never accessed here.
32 unsafe impl Send for MemoryLease {}
33 unsafe impl Sync for MemoryLease {}
34
35 impl MemoryLease {
36     /// Export ownership through callbacks compiled in the issuing library.
37     pub fn new<T: Send + Sync + 'static>(owner: Arc<T>, bytes: u64) -> Self {
38         unsafe extern "C" fn retain<T>(opaque: *const c_void) {
39             // SAFETY: only new() constructs this token, from Arc<T>::into_raw.
40             unsafe { Arc::<T>::increment_strong_count(opaque.cast::<T>()) };
41         }
42         unsafe extern "C" fn release<T>(opaque: *const c_void) {
43             // SAFETY: each token owns one reference created by new()/retain().
44             unsafe { drop(Arc::<T>::from_raw(opaque.cast::<T>())) };
45         }
46         Self {
```

```

47         header: Header {
48             version: 1,
49             size: std::mem::size_of::<Self>() as u32,
50         },
51         bytes,
52         opaque: Arc::into_raw(owner).cast::<c_void>(),
53         retain: retain::<T>,
54         release: release::<T>,
55     }
56 }
57
58 /// Validate the ABI and quota, then take an independently releasable reference.
59 ///
60 /// # Safety
61 /// `pointer` must address a readable, aligned header. A matching header must
62 /// address a live MemoryLease with valid thread-safe, non-unwinding callbacks.
63 /// Its issuing library must remain loaded until every imported clone is gone.
64 pub unsafe fn import(
65     pointer: NonNull<c_void>,
66     expected_bytes: u64,
67 ) -> Result<Self, &'static str> {
68     // SAFETY: the caller guarantees at least the fixed header is readable.
69     let header = unsafe { pointer.cast::<Header>().as_ref() };
70     if header.version != 1 || header.size as usize != std::mem::size_of::<Self>()
71 {
72         return Err("native memory lease ABI mismatch");
73     }
74     // SAFETY: the validated header and caller's capsule contract cover Self.
75     let lease = unsafe { pointer.cast::<Self>().as_ref() };
76     if lease.bytes != expected_bytes || lease.opaque.is_null() {
77         return Err("native memory lease quota mismatch");
78     }
79     Ok(lease.clone())
80 }
81
82 pub fn bytes(&self) -> u64 {
83     self.bytes
84 }
85
86 impl Clone for MemoryLease {
87     fn clone(&self) -> Self {
88         // SAFETY: self owns a live reference and callbacks obey the ABI contract.
89         unsafe { (self.retain)(self.opaque) };
90         Self {
91             header: Header {
92                 version: self.header.version,
93                 size: self.header.size,
94             },
95             bytes: self.bytes,
96             opaque: self.opaque,
97             retain: self.retain,
98             release: self.release,
99         }
100     }
101 }
102
103 impl Drop for MemoryLease {
104     fn drop(&mut self) {
105         // SAFETY: this instance owns exactly one reference, relinquished once.
106         unsafe { (self.release)(self.opaque) };
107     }
108 }
109
110 impl Debug for MemoryLease {
111     fn fmt(&self, f: &mut Formatter<'_>) -> std::fmt::Result {
112         f.debug_struct("MemoryLease")
113             .field("version", &self.header.version)
114             .field("bytes", &self.bytes)
115             .finish_non_exhaustive()
116     }
117 }
118
119 #[cfg(test)]
120 mod tests {
121     use std::sync::atomic::{AtomicUsize, Ordering};
122

```

```

123     use super::*;
124
125     struct Owner(Arc<AtomicUsize>);
126     impl Drop for Owner {
127         fn drop(&mut self) {
128             self.0.fetch_add(1, Ordering::SeqCst);
129         }
130     }
131
132     #[test]
133     fn imports_validate_before_clone_and_release_only_the_final_owner() -> Result<(),
134     &'static str>
135     {
136         let drops = Arc::new(AtomicUsize::new(0));
137         let exported = MemoryLease::new(Arc::new(Owner(drops.clone())), 64);
138         let pointer = NonNull::from(&exported).cast::<c_void>();
139         // SAFETY: the exported lease remains alive during both imports.
140         assert!(unsafe { MemoryLease::import(pointer, 65) }.is_err());
141         let imported = unsafe { MemoryLease::import(pointer, 64) }?;
142         drop(exported);
143         assert_eq!(drops.load(Ordering::SeqCst), 0);
144         let output_owner = imported.clone();
145         drop(imported);
146         assert_eq!(drops.load(Ordering::SeqCst), 0);
147         std::thread::spawn(move || drop(output_owner))
148             .join()
149             .map_err(|_| "release thread failed")?;
150         assert_eq!(drops.load(Ordering::SeqCst), 1);
151         let wrong = Header {
152             version: 2,
153             size: 8,
154         };
155         // SAFETY: a rejected version only reads the fixed header.
156         assert!(
157             unsafe { MemoryLease::import(NonNull::from(&wrong).cast::<c_void>(), 64) }
158             .is_err()
159         );
160         Ok(())
161     }

```

Resource ABI crate manifest

Source: [crates/sail-native-resource-ffi/Cargo.toml](#). 9 lines.

```

1 [package]
2 name = "sail-native-resource-ffi"
3 version = "0.1.0"
4 edition = "2024"
5 license = "Apache-2.0"
6 publish = false
7
8 [lints]
9 workspace = true

```

Complete local server startup commands

Source: [examples/extensions/TUTORIAL.md](#). 10 lines.

```

1 export PYTHONHOME="$(.venv/bin/python -c 'import sys; print(sys.base_prefix)')"
2 export PYTHONPATH="$(.venv/bin/python -c 'import sysconfig; print(sysconfig.get_paths(
3     )["purelib"])')"
4 export DYLD_LIBRARY_PATH="$(.venv/bin/python -c 'import sysconfig; print(sysconfig.get
5     _config_var("LIBDIR") or "")')"
6 export LD_LIBRARY_PATH="$DYLD_LIBRARY_PATH"
7 export SAIL_EXPERIMENTAL_EXTENSIONS=1
8 export SAIL_EXECUTION_DEFAULT_PARALLELISM=4
9 export SAIL_CLUSTER_WORKER_INITIAL_COUNT=2
10 export SAIL_CLUSTER_WORKER_MAX_COUNT=2
11 SAIL_MODE=local SAIL_EXPERIMENTAL_PROCESS_WORKERS=0 \
12     target/extensions-poc/host/debug/sail spark server --ip 127.0.0.1 --port 50051

```